CareerStar — Audience Q&A Prep

Simple spoken answers for likely audience questions · Capital One talk · August 6, 2026

The idea

Q1. How does it rate careers like stocks?

The same way an analyst rates a stock — buy, hold, or sell — but for careers. Every career gets a score out of 100 built from real government data about the job, plus a star rating. High score means the career's value should hold up; low score means AI can already do most of the work.

Q2. How does a career actually get rated?

Three ingredients: how fast the field is growing, how well it pays, and how much of the daily work AI can already do. Growth and pay push the score up; AI risk pulls it down. Then every career is ranked against all the others — five stars means it beat 90% of the field.

Q3. Does it really use stock math?

Yes — the core formula is risk-adjusted return, the same idea investors use. A big reward means less if the risk is huge, so the reward gets discounted in proportion to the risk. And the stars are graded on a curve exactly like Morningstar grades mutual funds.

Q4. Why did you build it?

Everyone my age is picking a career while AI reshuffles them, and the internet's answers are either pep talks or doom quizzes. I wanted the kind of answer an analyst gives: real numbers, named sources, and an honest 'it's close' when it's close. Nobody had built that, so I did.

Q5. Who is it for?

Students like me — people choosing a major or weighing a career switch. You type in the careers you're deciding between and it shows you the trade-offs side by side, with the reasoning visible.

Q6. Why call it CareerStar?

It's a nod to Morningstar, the company that rates mutual funds with stars. Same idea, different asset: instead of rating funds, I rate careers.

The math

Q7. What's the actual formula?

Four lines. Return is half growth, half pay. Risk is mostly AI exposure plus a little volatility. Value is return multiplied by (1 minus risk) — so risk discounts the reward. The final score is 70% that value, 30% how well the career fits your interests.

Q8. Why multiply by risk instead of subtracting it?

If you subtract, a big paycheck can hide a big risk — the career still scores fine. Multiplying discounts the reward in proportion to the risk, the way investors do it: double the risk, double the

haircut.

Q9. Who decided how much AI risk matters? Why 0.7?

I did — and I say so out loud. There's no answer key of correct career choices to tune against. So instead of pretending precision, the site re-runs every ranking 729 times with those numbers wiggled up and down, plus five rival formulas. When the answer flips, it says 'close call' instead of acting sure.

Q10. What are the five formulas?

Standard (my headline one — reward marked down by AI risk), a growth optimist that ignores AI, a defensive one that puts safety first, an efficiency one that asks for reward per unit of risk, and a plain average as the baseline. If my clever formula can't beat a plain average, I want to know.

Q11. How do the stars work?

A forced curve, borrowed from Morningstar: only the top 10% of careers get five stars, the next 22.5% get four, and so on. If most careers could get five stars, stars would mean nothing — scarcity is what makes them information.

Q12. What does 'fit' mean?

You tell the site your interests, and it compares them to the skills each job actually uses day to day, from a government database called O*NET. That's why two people can see different scores for the same job.

Q13. Isn't 'AI risk' just another word for 'declining job'?

Surprisingly, no. Across all 730 careers, AI exposure and projected growth are nearly unrelated — a job can be growing fast and still be very automatable. The AI number knows something growth doesn't.

Q14. How do you know the formula works?

I tested it on the past. I dug up the government's 2014 job projections from an internet archive, scored that job market with today's formula, and graded it against what really happened by 2024. It flagged 48% of the careers that truly declined — random guessing gets 33%.

Q15. Did anything beat your formula in that test?

Yes — the government's own plain projections, narrowly. Which makes sense: my AI ingredient had nothing to price in 2014, because modern AI didn't exist yet. I publish that result anyway. The awkward numbers earn more trust than the good ones.

The AI build

Q16. What's an agentic framework?

A regular AI chat is one assistant that does whatever you last asked. An agentic framework organizes AI into a team — several AI 'agents,' each with a specific job, a checklist, and rules for handing work to each other. Instead of one agreeable helper, I had a product manager agent, an

architect agent, a developer agent, and a squad of reviewers — a tiny software company where every employee is an AI and I'm the boss.

Q17. What does it mean that BMAD is 'spec-driven'?

The writing comes before the code. A spec — short for specification — is a written document saying exactly what you're building and how you'll know it's done. No agent writes code until the specs exist: the plan, the architecture rules, and the work broken into small tasks with done-checklists. In a chat, the plan drifts with every message; with a spec, the plan is a file — fixed, checkable, and shared by every agent. Vibe coding is 'prompt and hope.' Spec-driven is 'write the plan down first, and make every agent obey the paper.'

Q18. Are the seven rules BMAD's rules or yours?

Mine. BMAD's contribution is the process — plan first, build in small tasks, attack with reviewers. One step in that process is an architect agent writing down rules your specific project must never break. The seven are what came out of that step for CareerStar; a different project would get different rules.

Q19. What are the seven rules?

One: the whole site is a single app in one language — no second system to maintain. Two: all math lives in one module; the screen only displays numbers, so a wrong score has exactly one possible home. Three: the government datasets get combined into one file ahead of time — the live site never assembles data on the fly, so every result is reproducible. Four: the AI explains, never decides — no code path lets it produce or change a number. Five: anything a visitor types is data, never instructions — that's the security rule; it blocks 'ignore your rules' tricks and caps input length and request rates. Six: no accounts, no database — nothing you type is stored. Seven: hosting happens exactly one way, the AWS setup — no hand-built server to misconfigure.

Q20. Did AI write this whole thing? What did YOU do?

AI agents typed most of the code — I'm open about that. My job was everything around the typing: I set the plan, chose the formula, set the rules the code can't break, and refereed the AI reviewers who attack every change. The discipline is the part that's mine.

Q21. What is BMAD?

A free, open-source framework that turns AI into a small team — like a tiny software company where I'm the boss. A product-manager agent grilled me until we had a written plan, an architect agent set seven unbreakable rules, a developer agent built it in 28 small tasks, and a cast of reviewer agents argued over every change.

Q22. How is that different from just chatting with ChatGPT?

That's vibe coding — you prompt, paste, and hope. Here, a written plan exists before any code does, 35 automated checks re-run on every change, and separate AI reviewers exist purely to find problems. Same AI, very different results.

Q23. Could the AI just make up the scores?

No — that's the most important rule of the whole project: the AI never computes a number. The live site does plain arithmetic with a fixed, public formula. Same inputs, same score, every time — and anyone can redo the math by hand and catch me if it's wrong. AI helped write the recipe; it never gets to cook the numbers.

Q24. What did the AI get wrong along the way?

Plenty — that's why the reviewer agents exist. Best catch: my own methodology page claimed the fit score worked in a fancier way than the code actually did. The reviewers caught it, we rebuilt it properly, and the story of the catch is published on the site.

Q25. What's left behind besides the code?

A full paper trail: the written plan, the architecture doc with its seven rules, all 28 tasks with their done-checklists, and the reviewers' arguments — saved as files in the project. You can audit the process, not just the product.

The launch (AWS)

Q26. What does AWS actually do for you?

It's the computers my site runs on — I rent Amazon's instead of running my own. My code lives on GitHub; a service called Amplify watches it and rebuilds the live site every time I save a change; Route 53 is the internet's phone book that points ashleylibasci.com at those computers.

Q27. Do you manage a server?

Never — I've never even logged into one. Saving my code is the release: Amplify builds the site, hosts it, and swaps the new version in with no downtime. No 2 a.m. patching; Amazon handles the machines, I handle my code.

Q28. How would I put up my own site?

Four steps: put your code on GitHub, connect AWS Amplify to it once, buy a domain and point it with Route 53, and then — the step nobody puts in tutorials — test it like a stranger, from your phone, on cellular. The real internet behaves differently than your laptop.

Q29. What was the hardest bug?

My spam blocker never blocked anyone. It passed every test at home, then in production, 30 rapid-fire requests sailed through. Turns out Amazon's own machinery sits between visitors and my site and stamps its address exactly where my code was reading — I was rate-limiting Amazon. The fix: read the right spot, add a backstop cap, and pin it down with an automated test.

Q30. What does it cost to run?

Almost nothing at this scale. The domain is the biggest line item — about the price of a pizza per year. That's part of why this setup is great for a student project.

Trust and what's next

Q31. Should I actually trust it with my career decision?

Trust it the way you'd trust an analyst report, not an oracle. Every input is public data you can check, the reasoning is shown, and the site tells you when a ranking is a close call. It's meant to sharpen your thinking, not replace it.

Q32. Where does the data come from?

The U.S. government, mostly — nothing I could fake. The Bureau of Labor Statistics covers about 730 occupations: headcount, projected growth, and pay. O*NET describes the skills each job uses. And a published research study estimated, job by job, how much of the work today's AI could already do.

Q33. Do you store what I type?

No. No accounts, no database — nothing you type is stored. Anything a visitor enters is treated as data, never as instructions, which also blocks the classic 'ignore your rules' trick.

Q34. What happens to this project next?

The model makes bets about the next decade — so in 2034 I'll grade them against what really happened and publish the results, hits and misses. Ratings agencies should eat their own cooking.

Q35. What would you do differently?

Test in production sooner. My best lesson of the summer — the spam-blocker bug — only existed on the real internet. Everything passed on my laptop; the real world is the test that counts.